Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

# Design and Implementation of a Real-Time Rendering Pipeline for a 3D Game Engine

Radoslaw Rodak B.Sc. Software Development, ATU Galway
GitHub: https://github.com/ATU-Game-Engine/GameEngine

## Introduction

This project involved the design and implementation of a real-time rendering pipeline as part of a custom 3D game engine built in C++. The engine was developed collaboratively as a Final Year Project, with each team member responsible for a distinct subsystem, rendering, framework architecture, and physics integration.

The rendering pipeline supports Phong lighting, shadow mapping, normal mapping, specular mapping, a cubemap skybox, and .obj model loading, all driven by external GLSL shaders managed through dedicated ShaderManager and TextureManager systems.

## System Overview

The engine is built in C++ using OpenGL for graphics, Bullet Physics for simulation, and Dear ImGui for the editor interface. It follows a component-based architecture split across three subsystems developed in parallel by the project team.

Brian Walsh designed the core framework and entity management system. Liam Ryan integrated the Bullet Physics engine for rigid body simulation and collision detection. The rendering pipeline, covered in this poster, handles all visual output, sitting on top of the framework and communicating with the scene through a serialisation system shared across all three subsystems.

## Tech Stack

- Language: C++
- Graphics API: OpenGL
- Shading: GLSL
- Physics: Bullet Physics
- UI: Dear ImGui
- Model Loading: Assimp, tinyobjloader
- Build System: CMake / vcpkg
- Version Control: GitHub

All rendering code is written from scratch with no third-party rendering framework. External libraries are limited to asset loading, physics, and UI. The core pipeline is entirely custom OpenGL.

## References

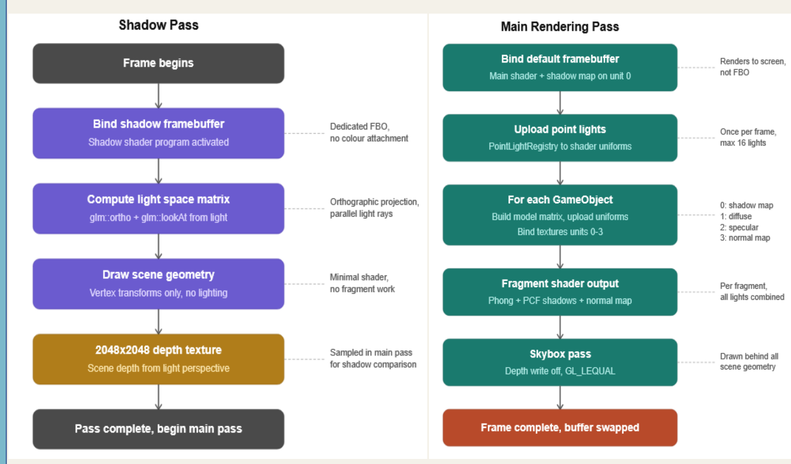
- LearnOpenGL — learnopengl.com
- OpenGL Documentation — docs.gl
- GLSL Specification — registry.khronos.org/OpenGL

## Rendering Pipeline

The rendering pipeline executes as a multi-pass system each frame. First, a shadow pass renders the scene from the light's perspective into a 2048 × 2048 depth framebuffer. The resulting depth texture is then sampled in the main pass to produce soft shadows using a PCF 3 × 3 kernel.

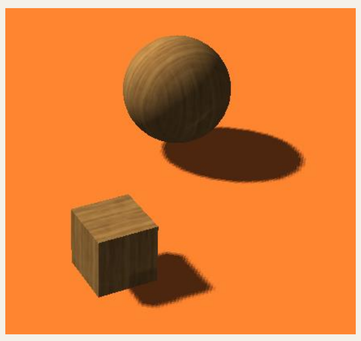
The main pass applies Phong lighting with support for one directional light and up to 16 point lights, managed through a PointLightRegistry singleton. Normal and specular maps are applied per-object using a full TBN matrix pipeline. Following the main pass, the skybox is rendered using a cubemap with depth testing set to GL_LEQUAL to ensure it always appears behind scene geometry.

Finally, an optional debug pass can be toggled with the V key, rendering wireframe colliders and light gizmos for editor use.

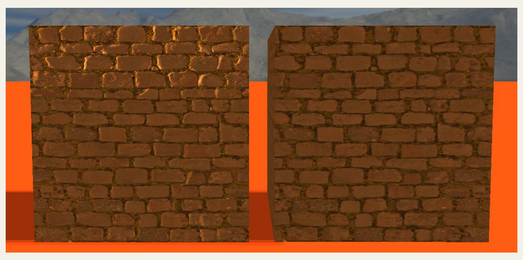



## Key Features

**Shadow Mapping**
A dedicated shadow pass renders scene depth from the light's perspective into a 2048 × 2048 FBO. Soft shadows are produced using PCF sampling across a 3 × 3 kernel.



**Normal Mapping**
Surface detail is enhanced using a full TBN matrix pipeline. Each fragment's normal is transformed from tangent space, giving the illusion of geometric detail without additional geometry.

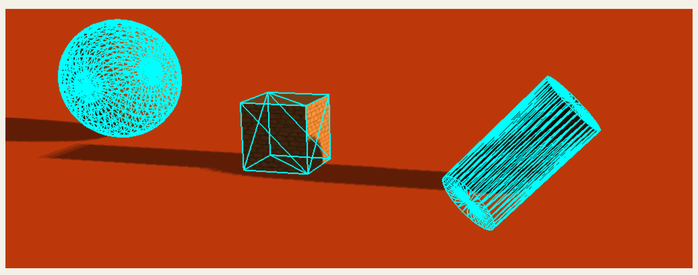


**Specular Mapping**
Surface shininess is controlled per-object using a specular map texture, allowing different materials to reflect light with varying intensity across the same mesh.

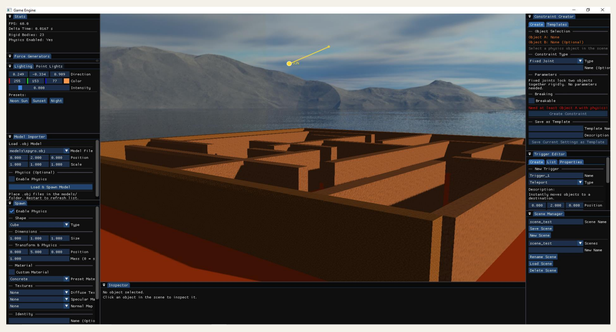
**Phong Lighting**
Supports one directional light and up to 16 dynamic point lights via a PointLightRegistry singleton, with smoothstep attenuation per light.

**Cubemap Skybox**
A seamless skybox rendered using a cubemap texture, always drawn behind scene geometry using GL_LEQUAL depth testing.

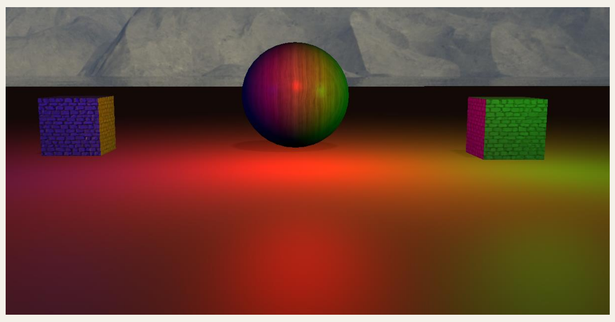
**Debug Visualisation**
Wireframe rendering and light gizmos can be toggled at runtime with the V key for editor use.



## Results

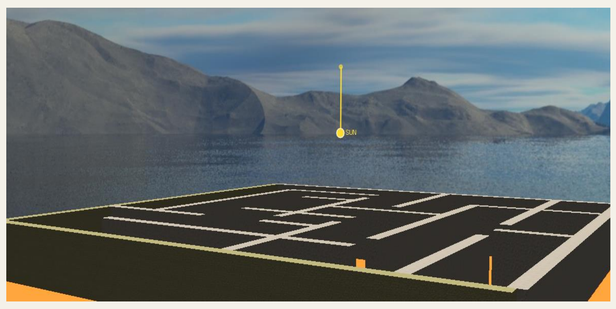


Full editor view showing the ImGui docking layout and scene controls



Multiple dynamic point lights rendered via PointLightRegistry

Spyro .obj model loaded with diffuse textures and Phong lighting



Cubemap skybox rendered using GL_LEQUAL depth testing, always drawn behind scene geometry

## Conclusion

This project successfully delivered a real-time rendering pipeline built from scratch in C++ and OpenGL. Core objectives were met including Phong lighting, PCF shadow mapping, normal and specular mapping, cubemap skybox, and .obj model loading with a ShaderManager and TextureManager system.

The most technically challenging aspect was the shadow mapping implementation, particularly resolving the noon sun bug. A degenerate glm::lookAt matrix when the light direction was straight down, requiring detection of the edge case and switching to an alternate up vector.

Future work includes omnidirectional point light shadows, PBR materials, and skeletal animation support. The engine serves as a solid foundation for further development.

## Acknowledgements

Supervised by Dr. Aaron Hurley, Atlantic Technological University Galway.